Lesson 4 — Insecure Cloud Configuration

Goal & Summary:

This vulnerability demonstrates insecure cloud configuration in which an S3 bucket allows file uploads that are automatically processed by a backend Lambda function. Because uploaded files are treated as trusted input, an attacker can inject arbitrary data into the system, potentially affecting backend execution.

Root Cause:

The vulnerability exists because the S3 bucket allows file uploads without strict access control or validation. Additionally, the Lambda function processes uploaded files without properly validating their content or structure, leading to unsafe handling of attacker-controlled input.

Environment:

1. AWS S3 bucket: dvsa-receipts-bucket-...
2. Lambda function: DVSA-SEND-RECEIPT-EMAIL
3. Tools: AWS Console, CloudWatch logs

Reproduction Steps:

1. Navigated to the S3 bucket used for receipt storage.
2. Observed that file upload functionality was available.
3. Created a test file (test.txt) locally.
4. Uploaded the file into the S3 bucket.
5. Observed that the upload triggered the Lambda function automatically.
6. Checked CloudWatch logs for the Lambda function.

Evidence & Proof :

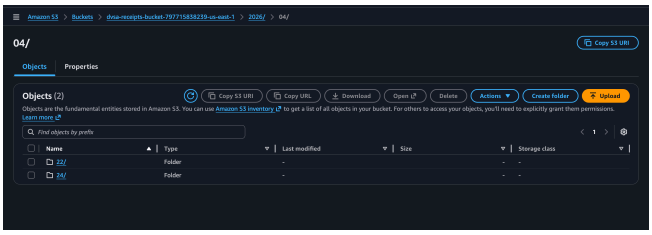
After uploading a test file into the S3 bucket, the backend Lambda function (DVSA-SEND-RECEIPT-EMAIL) was automatically triggered, as shown by the START RequestId log entry.

The CloudWatch logs show that the function attempted to process the uploaded file and produced errors related to email sending and invalid input values. This confirms that the Lambda function directly processes uploaded objects without strict validation.

This demonstrates that an attacker can upload arbitrary files into the S3 bucket and influence backend processing, proving the existence of an insecure cloud configuration vulnerability.

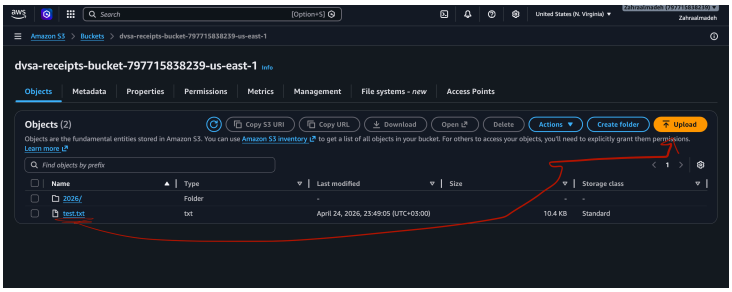
Screenshots to include:

1. S3 bucket

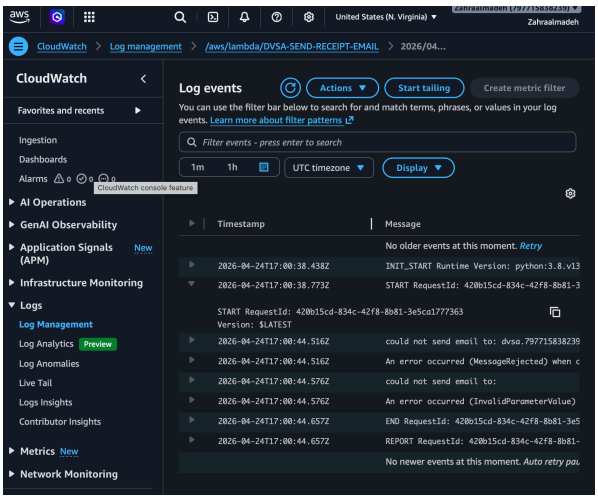


2. Upload screen

```
    ]
  }
[zm@Zs-MacBook-Pro ~ % echo "test file from attacker" > test.txt
```



3. CloudWatch logs



- show:
    - START RequestId
    - error messages

Fix Strategy:

The vulnerability was mitigated by applying both configuration-level and application-level protections.

First, the S3 bucket policy was updated to restrict access to only the Lambda execution role, removing broad or overly permissive access. This ensures that only authorized services can interact with the bucket.

Second, input validation was added to the Lambda function to verify file names and file types before processing. Only expected .raw files with safe naming patterns are allowed, and all other files are rejected.

These changes enforce the principle of least privilege and prevent unsafe processing of attacker-controlled input.

Config Changes:

The S3 bucket policy was modified to restrict access to the specific Lambda execution role, eliminating overly permissive access.

The Lambda function (send_receipt_email.py) was updated to include validation checks:

```python
import re


def is_valid_file(key):
    return re.match(r'^[a-zA-Z0-9._/-]+$', key)


def lambda_handler(event, context):
    file_key = event['Records'][0]['s3']['object']['key']
    file_key = urllib.parse.unquote_plus(urllib.parse.unquote(file_key))

    print("Processing file:", file_key)

    if not is_valid_file(file_key):
        print("Invalid file name detected")
        return {"status": "rejected"}

    if not file_key.endswith(".raw"):
        print("Invalid
        return {"status
```

AWS Lambda code editor based on Code-OSS (VS Code open source)

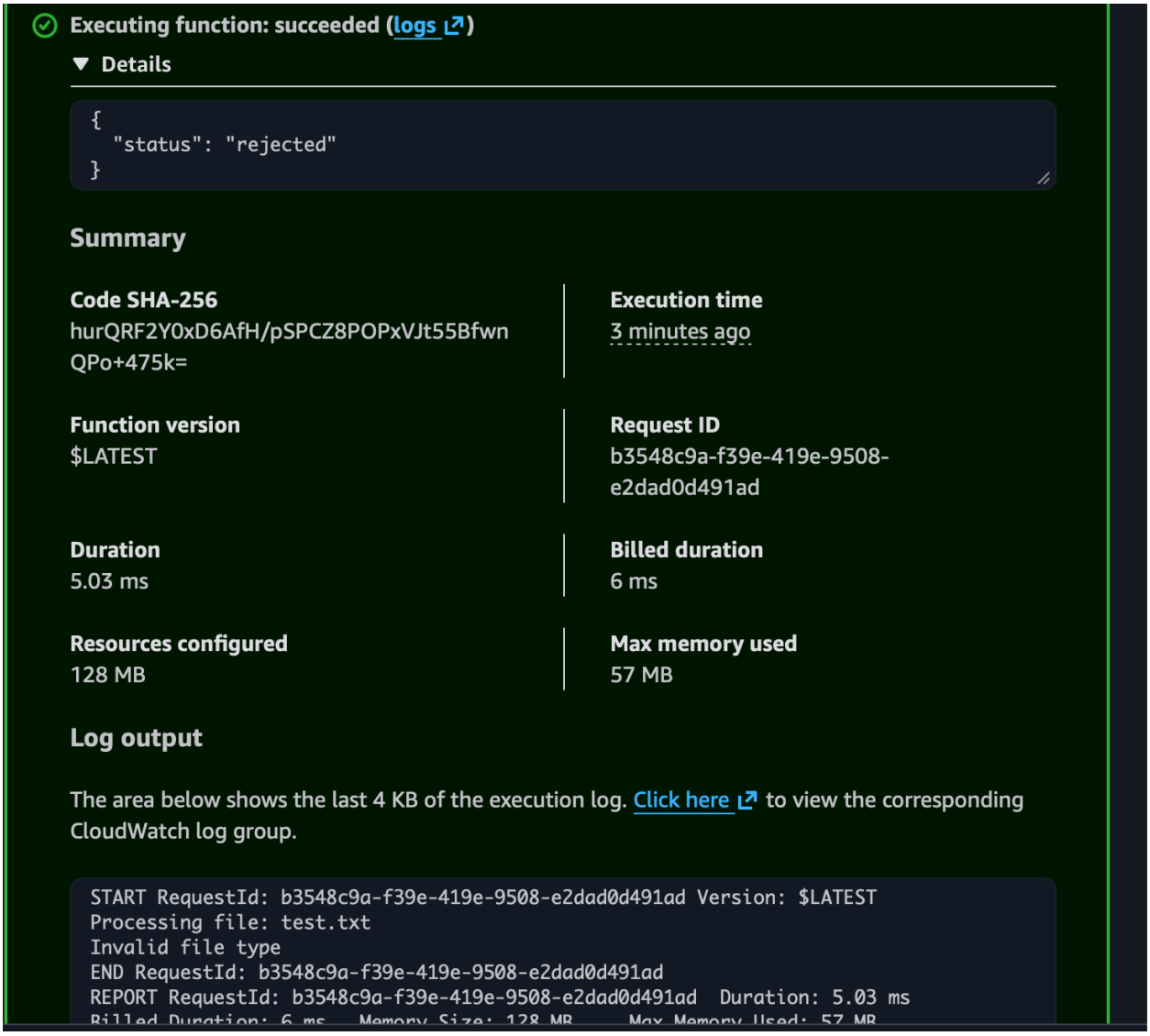
These checks ensure that only valid and expected files are processed.

Verification After Fix:

After applying the fix, the same test was performed using a .txt file. The Lambda function was triggered, but instead of processing the file, it rejected the input and returned a "status": "rejected" response.

CloudWatch logs confirmed that the function detected invalid file type and stopped execution. This demonstrates that unsafe inputs are no longer processed.

Valid .raw files continue to be processed correctly, confirming that the fix preserves intended functionality while blocking malicious inputs.



```
Status: Succeeded
Test Event Name: test1

Response:
{
  "status": "rejected"
}
```

Structured Analysis:

Table A:

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Insecure Cloud Configuration	Only authorized uploads should be accepted, and only safe files should be processed by backend services	S3 bucket view, CloudWatch logs, Lambda execution logs	System should only process valid receipt files generated internally	Uploaded arbitrary .txt file triggered Lambda and was processed, causing backend errors

Table B:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Insecure Cloud Configuration	Backend processed attacker-controlled files without validation, violating secure input handling and least privilege	Misconfiguration / security-relevant abuse	Restricted S3 bucket policy and added file validation in Lambda	Invalid files are now rejected, and Lambda no longer processes unsafe input

Lessons Learned:

This lesson demonstrates the importance of secure cloud configuration and input validation in serverless systems. Allowing unrestricted file uploads and processing them without validation introduces serious security risks.

In serverless architectures, services such as S3 and Lambda are tightly integrated, meaning that a misconfiguration in one component can directly impact backend logic. By enforcing least privilege and validating all external input, developers can prevent attacker-controlled data from influencing system behavior.

The key takeaway is that both cloud configuration and application logic must be secured together to prevent exploitation.